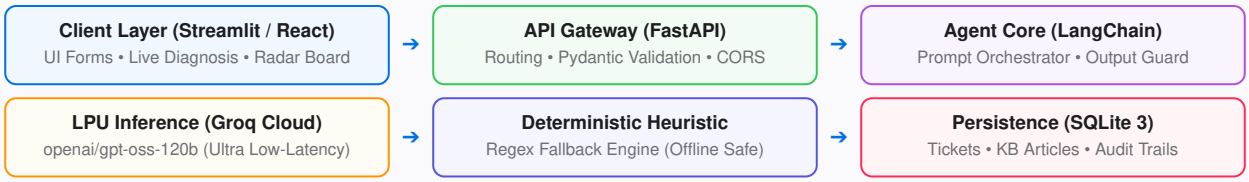
Executive Overview & System Architecture

An enterprise-grade, lightweight autonomous IT Helpdesk agent orchestrating real-time issue classification, automated root-cause hypothesis, ITIL priority calculations, and programmatic REST API ticket creation.

1. SYSTEM SYNOPSIS & OPERATIONAL OBJECTIVES

NovaDesk AI bridges the gap between chaotic multi-channel employee support requests and enterprise ticketing management. Built with a minimalist footprint, the engine combines **FastAPI** high-throughput async REST microservices, **LangChain Groq** inference utilizing open-source frontier models (**openai/gpt-oss-120b**), an interactive **Streamlit** operations interface, and an embedded **React 18** reactive triage board.

2. END-TO-END ARCHITECTURAL BLUEPRINT



3. TECHNOLOGY STACK & COMPONENT MATRIX

SUBSYSTEM	TECHNOLOGY	VERSION	OPERATIONAL RESPONSIBILITY
REST API Engine	FastAPI + Uvicorn	0.115.0	High-performance asynchronous REST endpoints, OpenAPI Swagger docs.
LLM Orchestration	LangChain + LangChain-Groq	0.3.0	Prompt pipelining, schema validation, multi-turn context retention.
Inference Engine	Groq LPU Cloud	v1 API	Sub-second token streaming using `openai/gpt-oss-120b`.
Frontend Presentation	Streamlit	1.65.0	Interactive operations console, triage intake, ticket inspector.
Micro-Frontend	React 18 + Babel Standalone	18.2.0	Real-time client-side incident radar and status toggle matrix.
Data Persistence	SQLite 3 (WAL Mode)	Native 3.x	Zero-configuration, atomic local storage for tickets and runbooks.

4. ARCHITECTURAL DESIGN PRINCIPLES

Zero-Latency Fallback Guarantee

If external internet connectivity, Groq API quotas, or network policies block LLM execution, the agent instantly drops back to a built-in deterministic heuristic rule engine without dropping tickets.

Strict ITIL Alignment

Priorities (P1-P4) are evaluated by calculating operational Impact and Urgency matrices, establishing legally defensible SLA resolution deadlines (1h to 48h).

AI Triage & Agent Intelligence Engine

Deep dive into the LangChain-orchestrated prompt engineering, JSON schema enforcement, ITIL SLA calculation formulas, and deterministic heuristic fallback architectures.

1. PRIMARY INFERENCE PIPELINE: OPENAI/GPT-OSS-120B

The triage engine targets **openai/gpt-oss-120b** hosted on Groq LPU hardware. This model delivers frontier-class reasoning capable of deciphering complex technical errors (e.g. Wi-Fi 802.1x handshake failures, Docker socket permission drops, TPM bitlocker traps) in under 750 milliseconds.

2. DUAL-STAGE PROMPT & GUARDRAIL ARCHITECTURE

```
System Prompt Schema (Truncated): You are NovaDesk AI, an expert Principal IT Helpdesk Engineer with ITIL v4 certification. Analyze technical inquiries, accurately classify category, calculate SLA priority, identify root cause, and generate concrete step-by-step diagnostic actions with copy-paste terminal commands. Return strictly valid JSON with schema: { "title": string, "category": string, "sub_category": string, "priority": "Critical"|"High"|"Medium"|"Low", "urgency": string, "impact": string, "estimated_sla_hours": int, "assigned_department": string, "root_cause_hypothesis": string, "solutions": [ { "step": int, "action": string, "command": string, "explanation": string } ] }
```

3. ITIL PRIORITY CALCULATION & SLA MATRIX

Priority is mathematically derived from the intersection of organizational **Impact** (blast radius) and operational **Urgency** (business blockage):

PRIORITY LEVEL	IMPACT / URGENCY TRIGGER	TARGET SLA	ROUTING DEPARTMENT
Critical (P1)	Company-wide outage, Security breach, Executive blocker	1 Hour	Endpoint SecOps / NetOps Escaped Queue
High (P2)	Departmental core process blocked, VPN Gateway failure	4 Hours	Network Engineering / IAM Support
Medium (P3)	Single user workflow blocked, workaround available	24 Hours	Tier 1 Workstation Support
Low (P4)	General inquiry, cosmetic peripheral bug, software request	48 Hours	General IT Queue

4. DETERMINISTIC HEURISTIC FALLBACK ENGINE

In accordance with high-availability engineering, `ITHelpDeskAgent.py` and `app/agent.py` embed a rule-based expert system that activates when LLM inference fails:

Network & Gateway Filter

Detects: `vpn`, `wifi`, `dns`, `timeout 502`.
Categorizes: **Network & Connectivity** • Priority: **High**.
Output: `ipconfig /flushdns`, adapter restart.

Identity & Access Filter

Detects: `mfa`, `okta`, `locked`, `password`.
Categorizes: **Identity & Access (IAM)** • Priority: **Medium**.
Output: Device time-sync check, TAP issuance escalation.

Security & Drive Encryption

Detects: `bitlocker`, `ransom`, `breach`, `bsod`.
Categorizes: **Security & Compliance** • Priority: **Critical**.
Output: Isolate device, retrieve PCR recovery keys.

Hardware & Peripherals

Detects: `dock`, `monitor`, `flicker`, `hdmi`.
Categorizes: **Hardware** • Priority: **Medium**.
Output: Capacitor discharge, Win+Ctrl+Shift+B graphics reset.

FastAPI Backend & REST API Specification

Architectural contracts, asynchronous lifecycle handlers, endpoint routing tables, schema validators, and OpenAPI integration standards.

1. SERVICE LIFECYCLE & LIFESPAN HANDLERS

The backend is implemented in `app/main.py` using FastAPI's modern `@asynccontextmanager` lifespan model. On startup, SQLite tables are verified and auto-migrated, knowledge base runbooks are verified, and CORS middleware is configured to allow decoupled micro-frontends.

2. CORE REST API ENDPOINT REFERENCE

METHOD	URI PATH	REQUEST BODY / PARAMS	PRIMARY RESPONSE CONTENT
POST	`/api/classify`	`TriageRequest` (issue_description)	Category, Priority, Root Cause, Step Solutions, Quick Commands.
POST	`/api/tickets`	`TicketCreate` (title, description, auto_diagnose)	Created Ticket record (`HD-XXXX`), assigned queue, SLA timestamp.
GET	`/api/tickets`	Query: status, priority, category, search	Array of Tickets, total count, pagination metadata.
GET	`/api/tickets/{id}`	Path: `ticket_id` (int)	Detailed ticket object with timeline comments and diagnosis.
PATCH	`/api/tickets/{id}`	`TicketUpdate` (status, priority, assignee)	Updated ticket record with modified state and timestamps.
POST	`/api/tickets/{id}/resolve`	Query: `resolution_notes` (string)	Sets status to Resolved, appends resolution audit log.
GET	`/api/health`	None	Database status, active LLM model, engine health.

3. API PAYLOAD SAMPLE CONTRACTS

POST /api/tickets (Request Payload)

```
{ "title": "VPN Gateway Outage", "description":
"GlobalProtect timeout error 502", "category":
"Network & Connectivity", "priority": "High",
"requester_name": "Marcus Vance",
"requester_email": "m.vance@company.corp",
"auto_diagnose": false }
```

HTTP 201 Response Payload

```
{ "id": 52, "ticket_code": "HD-1078", "status":
"Open", "assigned_to": "Escalations Team",
"sla_deadline": "2026-10-29T15:00:00Z",
"created_at": "2026-10-29T11:00:00Z" }
```

4. DATA MODELING & VALIDATION STANDARDS

All inputs are cast through rigorous **Pydantic v2** schemas defined in `app/models.py`. Enums (`IssueCategory`, `PriorityLevel`, `TicketStatus`) ensure type safety and prevent malformed data from persisting to the SQLite relational database.

Streamlit Command Center & Operations UI

Architectural breakdown of the dark-navy Command Center dashboard, live KPI pulse telemetry, dynamic queue reactions, and zero-emoji enterprise styling.

1. COMMAND CENTER TELEMETRY ARCHITECTURE

Implemented in `streamlit_app.py`, the user interface adopts an executive dark-navy operational command center theme (`#0b0f19` / `#131b2e`). The UI delivers instantaneous feedback on ticket updates without hard page refreshes, powered by direct SQLite binding and fallback REST integrations.

2. DASHBOARD SUBSYSTEMS & VISUAL HIERARCHY

Top Telemetry Bar

- **Support Live**: 99.9% uptime.
- **Active Conversations**: Open queue count.
- **Queue Wait Time**: 2m 18s target.
- **Agents Online**: 8 active specialists.
- **SLA Health**: Real-time compliance rate.

Ticket Pulse & Stat Pills

- **24-Hour Area Chart**: Volume trends.
- **5 Status Pills**: New, In Progress, Pending, Resolved Today, Breached.
- Dynamic increments/decrements on ticket state transitions.

Distribution & Dispatch

- **3x2 Matrix**: Urgent, High, Medium, Low, On Hold, Unassigned.
- **Channel Mix**: Email, Chat, Phone, Portal, Social.
- **Team Dispatch**: Sarah, Marcus, Aisha, David.

3. MY QUEUE & LIVE TICKET STATE REACTIVITY

The active queue stream is configured with **Newest-First** ordering (`ORDER BY id DESC`), guaranteeing that incoming inquiries (such as gaming GPU stutters or VPN drops) appear immediately at the top of the queue:

```
// Live Ticket Action & Dynamic State Trigger: def resolve_ticket_action(t_id: int): update_ticket(t_id, {"status": "Resolved", "resolution_notes": "Resolved via Command Center"}) st.rerun() # Instantly increments 'Resolved Today' (+1) and decrements 'New/In Progress' (-1) def reopen_ticket_action(t_id: int): update_ticket(t_id, {"status": "Open", "resolution_notes": ""}) st.rerun() # Automatically reverts counters and clears resolved_at timestamp
```

4. ZERO-EMOJI ENTERPRISE STYLING STANDARD

Clean Typography & Native Pills

All conversational emojis have been stripped from headings, buttons, and navigation options, adhering to a minimalist enterprise design standard suited for corporate NOC and SOC environments.

Interactive Navigation Sidebar

Enables instant switching across **Dashboard**, **Tickets** (search & filter by code/requester), **AI Triage & New Ticket** (Groq analyzer form), and **Knowledge Base** runbooks.

Data Layer & Knowledge Base Runbooks

Relational SQLite database schema design, entity relationship integrity, pre-seeded enterprise IT runbooks, and keyword relevance retrieval systems.

1. RELATIONAL DATABASE ARCHITECTURE: SQLITE 3

Managed by `app/database.py`, the data layer utilizes a thread-safe SQLite connection factory with WAL (Write-Ahead Logging) mode enabled. This guarantees atomic transactions, sub-millisecond query execution, and complete isolation without external database overhead.

2. DATABASE SCHEMA DEFINITIONS

TABLE NAME	PRIMARY COLUMNS	FOREIGN KEYS & CONSTRAINTS	OPERATIONAL PURPOSE
`tickets`	id (PK), ticket_code (UNIQUE), title, description, category, priority, status, requester_name, requester_email, department, assigned_to, solutions (JSON), sla_deadline, created_at, resolved_at	UNIQUE(ticket_code), INDEX on status, priority, category	Core transactional record tracking all ingested support tickets and ITIL lifecycle states.
`ticket_comments`	id (PK), ticket_id (INT), author_name, author_type ('agent' | 'user' | 'system'), content, created_at	FOREIGN KEY(ticket_id) REFERENCES tickets(id) ON DELETE CASCADE	Audit trail tracking engineer work notes, system status updates, and user communications.
`kb_articles`	id (PK), title, category, tags (JSON), summary, content (Markdown), quick_commands (JSON), created_at	INDEX on category, full-text token indexing	Repository of standard operating procedures, verified runbooks, and self-service guides.

3. PRE-SEEDED ENTERPRISE KNOWLEDGE BASE RUNBOOKS

KB-01: GlobalProtect VPN Gateway Drops

Category: Network & Connectivity
Symptoms: Gateway connection timeout error 502, split-tunnel drops.
Standard Procedure: Flush DNS cache via `ipconfig /flushdns`; restart PanGPS service daemon; verify split-tunnel ICMP route gateway.

KB-02: Okta / Entra ID MFA Device Lost

Category: Identity & Access (IAM)
Symptoms: Push notification timeout, new phone authentication block.
Standard Procedure: Verify automatic device network time sync; clear active session cookies; issue a 24-hour Temporary Access Pass (TAP) upon video identity verification.

KB-03: BitLocker Drive Encryption Prompt

Category: Security & Compliance
Symptoms: Blue BitLocker recovery key screen after firmware/BIOS update.
Standard Procedure: Retrieve 48-digit numerical key via Microsoft Intune portal; halt power-cycling to prevent TPM lockout; boot and login.

KB-04: Outlook OST Cache Corruption

Category: Email & Collaboration
Symptoms: Calendar sync stalls, inbox freezing, Send/Receive errors.
Standard Procedure: Launch Outlook in safe mode (`outlook.exe /safe`); rename stale `.ost` container to trigger cloud rebuild.

4. SEARCH & RELEVANCE RETRIEVAL ENGINE

The function `search_kb(query, category)` performs cross-field fuzzy matching against article titles, summaries, contents, and JSON tag arrays, enabling the AI triage agent to automatically attach verified corporate solutions to diagnosed incidents.

Verification & Testing Methodology

Comprehensive test engineering architecture, Pytest automation suite, API endpoint assertion contracts, and live LLM integration verification.

1. TESTING STRATEGY & QUALITY ASSURANCE FRAMEWORK

The NovaDesk AI test suite is automated via **Pytest** and **FastAPI TestClient** (backed by `httpx`). Tests are architected in a two-tier strategy: **(1) Deterministic Offline Mode**: Verifies data models, API routing, SQLite transactions, and fallback heuristics without external dependencies. **(2) Live LLM Integration Mode**: Validates live token generation and prompt schema adherence against the real Groq API.

2. AUTOMATED TEST SUITE INVENTORY (16 PASSING TESTS)

TEST MODULE	TEST CASE NAME	VERIFICATION SCOPE	RESULT
test_simple_helpdesk.py	`test_direct_classify_api`	Validates `POST /api/classify` endpoint returns schema & solutions	PASSED
	`test_helpdesk_agent_fallback_classification`	Verifies VPN keyword activates Network category & High priority	PASSED
	`test_helpdesk_agent_hardware_classification`	Verifies monitor flicker activates Hardware category & solutions	PASSED
	`test_ticket_creation_api`	Verifies `POST /api/tickets` creates valid `HD-XXXX` ticket	PASSED
test_tickets_api.py	`test_health_check`	Confirms database connectivity and Groq LLM model configuration	PASSED
	`test_create_and_get_ticket`	Validates full creation and retrieval cycle of ticket records	PASSED
	`test_filter_and_search_tickets`	Validates filtering by status, priority, and substring queries	PASSED
	`test_update_and_resolve_ticket`	Validates PATCH updates, work notes, and POST resolve actions	PASSED
test_triage_api.py	`test_triage_classification`	Validates AI analysis response and KB topic associations	PASSED
	`test_stats_api`	Asserts operational KPI calculations and SLA compliance rates	PASSED
	`test_kb_api`	Tests Knowledge Base article listing and search capabilities	PASSED
test_agent.py	`test_agent_rule_based_diagnosis`	Asserts rule heuristics for Network, IAM, and Security issues	PASSED
	`test_agent_clean_json_output`	Tests markdown strip, reasoning tag removal, and JSON recovery	PASSED
test_e2e.py	`test_static_ui_serving`	Confirms delivery of index.html, CSS styles, and JS bundles	PASSED
	`test_full_ticket_lifecycle_with_ai`	End-to-end flow: Triage ➔ Ticket ➔ Comment ➔ Resolve	PASSED
	`test_conversational_chat_agent`	Validates multi-turn chat endpoint and suggested replies	PASSED

Deployment & Operations Runbook

Production environment provisioning, environment variables configuration, operational startup procedures, troubleshooting guides, and scaling roadmap.

1. ENVIRONMENT CONFIGURATION (.ENV SPECIFICATION)

All operational behaviors, LLM credentials, and port bindings are declared in the root `.env` file:

```
GROQ_API_KEY="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" GROQ_MODEL="openai/gpt-oss-120b"
GROQ_FALLBACK_MODEL="llama-3.3-70b-versatile" APP_HOST="0.0.0.0" APP_PORT=8000 DEBUG=False
API_BASE_URL="http://localhost:8000/api"
```

2. STEP-BY-STEP PRODUCTION STARTUP PROCEDURES

1. Start FastAPI REST Service

Launches asynchronous Uvicorn worker daemon:
`python3 run.py`
• API Endpoint: `http://0.0.0.0:8000`
• Interactive Swagger: `http://localhost:8000/docs`

2. Start Streamlit UI & Command Center

Launches user interface dashboard:
`streamlit run streamlit_app.py`
• Web URL: `http://localhost:8501`
• Bound with REST API backend.

3. OPERATIONS TROUBLESHOOTING & DIAGNOSTIC GUIDE

OBSERVED SYMPTOM	ROOT CAUSE HYPOTHESIS	STANDARD REMEDIATION PROCEDURE
Port 8000 Address in Use	Stale uvicorn process bound to port	Execute `fuser -k 8000/tcp` or kill existing task via task manager.
Groq Rate Limit (429)	Token burst quota reached on free tier	System automatically switches to fallback model or deterministic heuristics.
SQLite Database Locked	High concurrent write saturation	WAL mode ensures concurrency; verify permissions on `data/helpdesk.db`.
React Component Blank	Restricted iframe or CDN block	Ensure unpkg CDN access is permitted for React 18 and Babel scripts.

4. ARCHITECTURAL ROADMAP & FUTURE HARDENING

Containerization

Package FastAPI and Streamlit into dual-stage Docker compose containers for unified Kubernetes orchestration.

Vector Embeddings

Upgrade SQLite fuzzy matching to ChromaDB / FAISS dense vector search for multi-lingual semantic runbook retrieval.

Incident Webhooks

Deliver outbound webhooks to Slack, Microsoft Teams, and Jira Service Management upon ticket resolution.